Анализ сложности алгоритмов

Пояснения к программе Alg5.cpp для защиты лабораторной работы

Программа продолжает практики Alg3 (простые сортировки) и Alg4 (слияние, Хоар, пирамида). Из них взяты сортировка вставками и сортировка слиянием. Все функции сделаны **шаблонными** и запускаются на больших массивах `int` и `double`, чтобы измерить среднее время работы и сравнить его со стандартной функцией `std::sort`.

Соответствие пунктам задания

Пункт задания	Где в программе
1. Сортировка $O(n^2)$ в виде шаблонной функции	`insertionSort` — сортировка вставками
2. Сортировка O(n log n) в виде шаблонной функции	`mergeSort` + `mergeSortRange` — сортировка слиянием
3. Программа для определения среднего времени (вместе с std::sort)	`measure`, `runTests`, `runAll`, `main`
4. Массивы, заполненные случайно	`fillRandom`
5. Массивы с упорядоченными фрагментами	`fillFragments`
6. Зависимости T(n)	файл results.csv и графики (раздел 10)
7. Схемы в UMLet	insertionSort.uxf, mergeSort.uxf — блок-схемы по ГОСТ (раздел 9)

Структура программы

Код идёт сверху вниз в таком порядке:

- подключение библиотек и константы;
- две сортировки: `insertionSort` и `mergeSort`, плюс обёртка `stdSort`;
- заполнение тестовых массивов: `fillRandom`, `fillFragments`;
- вспомогательные функции: `isSorted`, `printArray`;
- замер времени: `measure` (одна сортировка), `runTests` (среднее по 10 массивам) и `runAll` (все размеры для одного типа);
- `main` — запускает замеры для `int` и для `double` и пишет результаты в файл.

1. Библиотеки и константы

```
#include <iostream>
#include <fstream>
#include <algorithm>
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <iomanip>

using namespace std;

const int TESTS = 10;          // сколько разных массивов на каждый размер n
const int MAX_N2 = 50000;      // insertionSort (O(n^2)) запускаем только до этого n
const int PARTS = 10;          // на сколько упорядоченных фрагментов делим массив
```

Строка	Зачем нужна
`<iostream>`	`cout`, `endl` — вывод на экран
`<fstream>`	`ofstream` — запись результатов в файл results.csv
`<algorithm>`	`sort` — стандартная сортировка (с ней сравниваем)
`<chrono>`	точный замер времени: `chrono::steady_clock`
`<cstdlib>`	`rand`, `srand` — случайные числа; `exit` — аварийный выход
`<ctime>`	`time(0)` — текущее время, чтобы при каждом запуске числа были разные
`<iomanip>`	`setw` (ширина столбца) и `setprecision` (знаки после запятой) для таблицы
`using namespace std;`	чтобы писать `cout` вместо `std::cout`

Константы задают параметры эксперимента:

- `TESTS = 10` — сколько разных массивов генерируется для каждого размера n. Время усредняется по ним, поэтому оно и называется **средним**.
- `MAX_N2 = 50000` — сортировку вставками запускаем только до этого размера. Дальше она работает слишком долго: на миллионе элементов это заняло бы минуты.
- `PARTS = 10` — на сколько упорядоченных кусков делится массив в п. 5.

`const` означает, что значение нельзя изменить во время работы программы.

2. Сортировка вставками — insertionSort, $O(n^2)$

```
template <typename T>
void insertionSort(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        T n = arr[i];
        int j = i - 1;
        while (j >= 0 && n < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = n;
    }
}
```

Идея

Массив мысленно делится на две части. Левая часть `arr[0..i-1]` всегда уже отсортирована. Берём следующий элемент `arr[i]`, сдвигаем вправо всех его больших соседей слева и ставим элемент в освободившееся место. Так сортированная часть растёт на один элемент за шаг. Точно так же сортируют карты в руке.

Построчно

Код	Что делает
`template <typename T>`	функция шаблонная: `T` — любой тип элементов (int, double, string…)
`void insertionSort(T arr[], int size)`	принимает массив и его длину, ничего не возвращает: массив передаётся по адресу, поэтому сортируется исходный массив
`for (int i = 1; i < size; i++)`	с `i = 1`, потому что один элемент `arr[0]` уже отсортирован сам по себе
`T n = arr[i];`	запоминаем вставляемый элемент: при сдвигах его ячейка будет перезаписана
`int j = i - 1;`	`j` идёт по отсортированной части справа налево
`while (j >= 0 && n < arr[j])`	пока не дошли до начала массива и слева стоит элемент **больше** вставляемого
`arr[j + 1] = arr[j];`	сдвигаем больший элемент на одну позицию вправо
`j--;`	переходим к следующему элементу слева
`arr[j + 1] = n;`	цикл остановился: `arr[j] <= n` или `j = -1`. Место для `n` — сразу правее, в `j + 1`

Почему `j >= 0` стоит первым. Оператор `&&` вычисляется слева направо и останавливается на первом false. Когда `j = -1`, проверка `arr[-1]` уже не выполняется, и выхода за границу массива нет. Если поменять условия местами, программа будет читать чужую память.

Почему `n < arr[j]`, а не `<=`. Равные элементы не сдвигаются, поэтому их взаимный порядок сохраняется. Такая сортировка называется **устойчивой**.

Пример работы на массиве [5, 2, 4, 1]

Шаг	n = arr[i]	Что происходит	Массив после шага
i = 1	2	5 > 2 → сдвиг 5 вправо; j = −1 → ставим 2 в arr[0]	[2, 5, 4, 1]
i = 2	4	5 > 4 → сдвиг; 2 < 4 → стоп, ставим 4 в arr[1]	[2, 4, 5, 1]
i = 3	1	сдвигаем 5, 4, 2; j = −1 → ставим 1 в arr[0]	[1, 2, 4, 5]

Всего 5 сдвигов. Это ровно число **инверсий**, то есть пар, стоящих в неправильном порядке: (5,2), (5,4), (5,1), (2,1), (4,1). Время работы сортировки вставками пропорционально n + число инверсий.

Почему сложность $O(n^2)$

Внешний цикл выполняется n − 1 раз. Внутренний цикл для элемента с номером i делает от 0 до i сдвигов.

- **Худший случай** (массив по убыванию): каждый элемент едет в самое начало, сдвигов $1 + 2 + \ldots + (n-1) = n(n-1)/2 \approx n^2/2$.
- **Средний случай** (случайный массив): каждый элемент в среднем уходит на половину пути, получается $\approx n^2/4$.
- **Лучший случай** (массив уже отсортирован): `while` ни разу не выполняется, всего n − 1 сравнение, то есть O(n).

В O-нотации константы (1/2, 1/4) и младшие слагаемые отбрасываются, поэтому в среднем и худшем случае сложность **$O(n^2)$**. Дополнительная память O(1): нужна только переменная `n`.

Практический смысл: если n увеличить в 2 раза, время вырастет примерно в $2^2 = 4$ раза.

3. Сортировка слиянием — mergeSort, O(n log n)

```
template <typename T>
void mergeSortRange(T arr[], T buf[], int left, int right) {
    if (right - left <= 1) return;

    int center = (right - left) / 2 + left;

    mergeSortRange(arr, buf, left, center);
    mergeSortRange(arr, buf, center, right);

    int i = left;
    int j = center;

    for (int k = left; k < right; k++) {
        if (j == right || (i < center && arr[i] <= arr[j])) {
            buf[k] = arr[i++];
        }
        else buf[k] = arr[j++];
    }

    for (int k = left; k < right; k++) {
        arr[k] = buf[k];
    }
}
```

```
template <typename T>
void mergeSort(T arr[], int size) {
    T* buf = new T[size];
    mergeSortRange(arr, buf, 0, size);
    delete[] buf;
}
```

Идея: «разделяй и властвуй»

- **Разделяй:** делим массив пополам.
- **Властвуй:** рекурсивно сортируем каждую половину тем же алгоритмом.
- **Объединяй:** сливаем две отсортированные половины в одну. Смотрим на первые элементы обеих половин и каждый раз забираем меньший.

Рекурсия останавливается, когда в куске 0 или 1 элемент: такой кусок уже отсортирован.

Построчно: mergeSort

Код	Что делает
`T* buf = new T[size];`	выделяем вспомогательный массив (буфер) того же размера. Слияние нельзя сделать «на месте», нужен буфер. Выделяем его **один раз**, а не на каждом шаге рекурсии, это быстрее
`mergeSortRange(arr, buf, 0, size);`	сортируем весь массив: диапазон от 0 до size
`delete[] buf;`	освобождаем память, иначе будет утечка памяти

Построчно: mergeSortRange

Функция сортирует кусок массива `[left, right)`: `left` входит в кусок, `right` — нет. Это **полуинтервал**, так же устроены итераторы в STL. Длина куска равна `right - left`.

Код	Что делает
`if (right - left <= 1) return;`	база рекурсии: 0 или 1 элемент уже отсортирован
`int center = (right - left) / 2 + left;`	середина куска. Левая половина [left, center), правая [center, right)
`mergeSortRange(arr, buf, left, center);`	рекурсивно сортируем левую половину
`mergeSortRange(arr, buf, center, right);`	рекурсивно сортируем правую половину
`int i = left; int j = center;`	`i` — текущий элемент левой половины, `j` — правой
`for (int k = left; k < right; k++)`	`k` — куда в буфере кладём очередной элемент
`if (j == right || (i < center && arr[i] <= arr[j]))`	берём из **левой** половины, если правая закончилась **или** левая ещё не закончилась и её элемент не больше правого
`buf[k] = arr[i++];`	кладём `arr[i]` в буфер, потом `i` увеличивается (постфиксный ++)
`else buf[k] = arr[j++];`	иначе берём из правой половины
`for (...) arr[k] = buf[k];`	копируем слитый кусок из буфера обратно в массив

Почему `(right - left) / 2 + left`, а не `(left + right) / 2`. Результат одинаковый, но `left + right` на очень больших массивах может превысить максимум int (≈ 2.1 млрд) и стать отрицательным. Разность `right - left` не переполняется.

Почему `arr[i] <= arr[j]` с «меньше или равно». При равенстве берём элемент из левой половины, то есть тот, что стоял раньше. Так равные элементы не меняются местами, и сортировка **устойчивая**.

Почему `j == right` проверяется первым. Если правая половина закончилась, `arr[j]` — уже чужой элемент, сравнивать с ним нельзя. Благодаря `||` при `j == right` остальное условие не вычисляется. Аналогично `i < center` защищает от чтения за концом левой половины.

Пример работы на массиве [5, 2, 4, 1]

```
mergeSortRange(0, 4): center = 2
├─ mergeSortRange(0, 2): center = 1
│   ├─ (0, 1) — 1 элемент [5], return
│   ├─ (1, 2) — 1 элемент [2], return
│   └─ слияние [5] и [2]:  2 < 5 → берём 2; правая кончилась → берём 5  ⇒ [2, 5]
├─ mergeSortRange(2, 4): center = 3
│   └─ слияние [4] и [1]  ⇒ [1, 4]
└─ слияние [2, 5] и [1, 4]:
      k=0: 2 <= 1? нет → 1      k=1: 2 <= 4? да → 2
      k=2: 5 <= 4? нет → 4      k=3: j == right → 5      ⇒ [1, 2, 4, 5]
```

Почему сложность O(n log n)

Массив делится пополам до кусков из одного элемента. Число уровней деления — сколько раз n можно разделить на 2, то есть $\log_2 n$. На каждом уровне все слияния вместе проходят по каждому элементу ровно один раз, это ~n операций.

Уровень	Кусков	Размер куска	Работа на уровне
0	1	n	n
1	2	n/2	2 · n/2 = n
2	4	n/4	4 · n/4 = n
…	…	…	n
$\log_2 n$	n	1	n

Итого $n \cdot \log_2 n$. Формально: $T(n) = 2 \cdot T(n/2) + c \cdot n$, по основной теореме о рекуррентных соотношениях $T(n) =$ **O(n log n)**. Это верно **для любых данных**: деление всегда идёт пополам, худшего случая нет.

Дополнительная память — буфер на n элементов, то есть **O(n)**, плюс глубина рекурсии O(log n). Это плата за гарантированную скорость.

Практический смысл: если n увеличить в 2 раза, время вырастет чуть больше чем в 2 раза.

4. Стандартная сортировка — stdSort

```
template <typename T>
void stdSort(T arr[], int size) {
    sort(arr, arr + size);
}
```

`sort(arr, arr + size)` — функция из `<algorithm>`. Ей передаются указатель на начало массива и указатель на элемент **после последнего**: `arr + size`. Это тот же полуинтервал, что и в mergeSortRange.

Обёртка `stdSort` нужна, чтобы у всех трёх сортировок был одинаковый вид `(T arr[], int size)`. Тогда их можно передавать в одну и ту же функцию замера `measure`.

Внутри `std::sort` (и в Visual Studio, и в gcc) — **интроспективная сортировка** (introsort):

- в основном быстрая сортировка (как `hoareSort` из Alg4);
- если рекурсия становится слишком глубокой (плохие опорные элементы), переключается на пирамидальную (как `heapSort` из Alg4), чтобы не скатиться в $O(n^2)$;
- маленькие куски (порядка 16–32 элементов) досортировывает вставками: на малых n это быстрее всего.

Стандарт C++ гарантирует для `std::sort` сложность O(n log n).

5. Заполнение тестовых массивов

Обе функции заполнения тоже **шаблонные**: одна и та же функция заполняет и массив `int`, и массив `double`.

```
template <typename T>
void fillRandom(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = (T)rand() + (T)rand() / RAND_MAX;
    }
}
```

П. 4 задания. Каждый элемент — случайное число.

- `(T)rand()` — случайное целое (в Visual Studio от 0 до 32767), приведённое к типу T.
- `(T)rand() / RAND_MAX` — второе случайное число, делённое на максимум, — дробная часть от 0 до 1.
- Для **double** получается дробное число, например 12345.678.
- Для **int** деление целочисленное, дробная часть отбрасывается и почти всегда равна 0 — остаётся просто `rand()`.
- `(T)` — приведение типа: превращаем число в тот тип, с которым работает шаблон.

```
template <typename T>
void fillFragments(T arr[], int size) {
    fillRandom(arr, size);
    int len = size / PARTS;
    for (int p = 0; p < PARTS; p++) {
        int left = p * len;
        int right = (p == PARTS - 1) ? size : left + len;
        sort(arr + left, arr + right);
    }
}
```

П. 5 задания: массив, состоящий из упорядоченных фрагментов.

- Сначала заполняем массив случайно (`fillRandom`).
- Делим его на `PARTS = 10` кусков длиной `len = size / PARTS`.
- Каждый кусок `[left, right)` сортируем отдельно через `sort`.
- `(p == PARTS - 1) ? size : left + len` — тернарный оператор «условие ? если да : если нет». Последний кусок тянется до конца массива и забирает остаток от деления. Например, при size = 25: len = 2, куски [0,2), [2,4), …, [16,18) и последний [18,25).

Получается 10 отсортированных фрагментов, но между собой они перемешаны. Например: [2, 5, 7, 9 | 1, 3, 6, 8 | 0, 4, 10, 11 | …].

6. Вспомогательные функции

```
template <typename T>
bool isSorted(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        if (arr[i] < arr[i - 1]) return false;
    }
    return true;
}
```

Проверяет, что массив отсортирован: каждый элемент не меньше предыдущего. Если нашли пару «больший перед меньшим», сразу возвращаем false. Нужна, чтобы **доказать**, что сортировки работают правильно, а не просто быстро.

```
template <typename T>
void printArray(T arr[], int size) {
    cout << "[ ";
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) cout << ", ";
    }
    cout << " ]" << endl;
}
```

Печатает массив в виде [a, b, c]. Запятая ставится после каждого элемента, кроме последнего. Используется в начале `main` для демонстрации шаблонов.

```
void printHeader() {
    cout << setw(10) << "n" << setw(16) << "insertionSort"
         << setw(16) << "mergeSort" << setw(16) << "std::sort" << endl;
}
```

Печатает шапку таблицы: n, insertionSort, mergeSort, std::sort. Вынесена в отдельную функцию, потому что шапка печатается 4 раза (2 типа × 2 вида данных).

7. Замер времени

7.1. measure — время одной сортировки

```
template <typename T>
double measure(void (*sortFunc)(T[], int), T src[], T arr[], int size) {
    for (int i = 0; i < size; i++) arr[i] = src[i];

    auto start = chrono::steady_clock::now();
    sortFunc(arr, size);
    auto end = chrono::steady_clock::now();

    if (!isSorted(arr, size)) {
        cout << "ERROR: array is not sorted!" << endl;
        exit(1);
    }
    return chrono::duration<double, milli>(end - start).count();
}
```

Код	Что делает
`template <typename T>`	функция шаблонная: мерит сортировку массива любого типа — int или double
`void (*sortFunc)(T[], int)`	**указатель на функцию**: параметр, в который можно передать любую функцию вида `void f(T[], int)`. Передаём `insertionSort<T>`, `mergeSort<T>` или `stdSort<T>`, и один и тот же код мерит любую сортировку
`T src[]`	исходный массив, он не меняется
`T arr[]`	рабочая копия, её и сортируем
`for (...) arr[i] = src[i];`	копируем исходные данные. Копирование идёт **до** запуска таймера и в замер не входит
`auto start = chrono::steady_clock::now();`	засекаем время. `auto` — компилятор сам выводит тип переменной
`sortFunc(arr, size);`	вызываем ту сортировку, которую передали
`auto end = chrono::steady_clock::now();`	время окончания
`if (!isSorted(arr, size))`	проверка результата. Если массив не отсортирован — ошибка, программа завершается через `exit(1)`
`chrono::duration<double, milli>(end - start).count()`	разность времён переводим в миллисекунды как дробное число: `double` — тип числа, `milli` — единица (1/1000 секунды)

Зачем копировать массив. Все три алгоритма должны получить **одинаковые** данные. Если сортировать сам `src`, то второй алгоритм получит уже отсортированный массив, и сравнение будет нечестным.

Почему chrono, а не clock() (как в Alg1). В Windows `clock()` меряет с шагом около 1 мс. Сортировка 1000 элементов идёт около 0.1 мс, и `clock()` показал бы 0. `steady_clock` меряет с точностью до наносекунд и не зависит от перевода системных часов.

7.2. runTests — среднее время по TESTS массивам

```
template <typename T>
void runTests(const char* type, const char* name, void (*fill)(T[], int), int size,
              T src[], T arr[], ofstream& csv) {
    double tIns = 0, tMerge = 0, tStd = 0;
    bool doIns = size <= MAX_N2;

    for (int t = 0; t < TESTS; t++) {
        fill(src, size);
        if (doIns) tIns += measure(insertionSort<T>, src, arr, size);
        tMerge += measure(mergeSort<T>, src, arr, size);
        tStd += measure(stdSort<T>, src, arr, size);
    }
    tIns /= TESTS;
    tMerge /= TESTS;
    tStd /= TESTS;

    cout << setw(10) << size;
    if (doIns) cout << setw(16) << tIns;
    else cout << setw(16) << "-";
    cout << setw(16) << tMerge << setw(16) << tStd << endl;

    csv << type << ";" << name << ";" << size << ";";
    if (doIns) csv << tIns;
    csv << ";" << tMerge << ";" << tStd << endl;
}
```

Код	Что делает
`const char* type`	тип элементов ("int" или "double") — пишется в первый столбец CSV
`const char* name`	вид данных ("random" или "fragments") — второй столбец CSV
`void (*fill)(T[], int)`	ещё один указатель на функцию: `fillRandom<T>` или `fillFragments<T>`
`T src[], T arr[]`	исходный массив и рабочая копия того же типа T
`ofstream& csv`	файл передаётся **по ссылке** (&): поток нельзя копировать, и пишем в тот же открытый файл
`double tIns = 0, tMerge = 0, tStd = 0;`	суммы времени для трёх алгоритмов (время всегда double, даже если массив int)
`bool doIns = size <= MAX_N2;`	запускать ли сортировку вставками на этом размере
`for (int t = 0; t < TESTS; t++)`	10 раз: новый массив → замер всех трёх сортировок на нём
`tIns /= TESTS; ...`	делим сумму на 10 — получаем **среднее время**
`cout << setw(16) << ...`	печать строки таблицы, `setw(16)` — ширина столбца 16 символов
`csv << type << ";" << name << ...`	та же строка в файл, через «;», чтобы её открыл Excel

Каждый новый массив создаётся случайно, поэтому усреднение по 10 массивам сглаживает случайные колебания: «удачные» и «неудачные» массивы, работу других программ, кэш процессора.

7.3. runAll — все замеры для одного типа

```
template <typename T>
void runAll(const char* type, ofstream& csv) {
    int sizes[] = { 1000, 2000, 5000, 10000, 20000, 30000, 40000, 50000,
                    100000, 200000, 500000, 1000000 };
    const int sizeCount = sizeof(sizes) / sizeof(sizes[0]);
    const int maxSize = 1000000;

    T* src = new T[maxSize];
    T* arr = new T[maxSize];

    cout << endl << "Random arrays, " << type << ":" << endl;
    printHeader();
    for (int i = 0; i < sizeCount; i++) {
        runTests(type, "random", fillRandom<T>, sizes[i], src, arr, csv);
    }

    cout << endl << "Arrays with " << PARTS << " sorted fragments, " << type << ":" << endl;
    printHeader();
    for (int i = 0; i < sizeCount; i++) {
        runTests(type, "fragments", fillFragments<T>, sizes[i], src, arr, csv);
    }

    delete[] src;
    delete[] arr;
}
```

- Шаблонная функция: `runAll<int>` делает все замеры на массивах int, `runAll<double>` — на массивах double.
- `sizes[]` — размеры массивов от 1000 до 1 000 000.
- `sizeof(sizes) / sizeof(sizes[0])` — число элементов массива: общий размер в байтах делим на размер одного элемента. Если добавить размер в список, считать вручную не нужно.
- `new T[maxSize]` — массивы в динамической памяти. Миллион double — это 8 МБ, а обычный локальный массив такого размера переполнил бы стек (по умолчанию 1 МБ).
- Два цикла: сначала все размеры на случайных массивах (`fillRandom<T>`), потом на массивах с фрагментами (`fillFragments<T>`).
- `delete[] src; delete[] arr;` — освобождаем память.

7.4. main

```
int main() {
    srand((unsigned)time(0));

    // Проверка, что шаблоны работают с разными типами
    int a[] = { 2, 5, 7, 8, 1, 4, 3, 9, 6, 0 };
    double b[] = { 3.5, -1.2, 8.0, 0.5, 2.25 };
    insertionSort(a, 10);
    mergeSort(b, 5);
    cout << "insertionSort<int>: ";
    printArray(a, 10);
    cout << "mergeSort<double>:  ";
    printArray(b, 5);
    cout << endl;

    ofstream csv("results.csv");
    csv << "type;data;n;insertionSort;mergeSort;std::sort" << endl;
    cout << fixed << setprecision(3);
    csv << fixed << setprecision(3);

    cout << "Average time, ms (" << TESTS << " arrays for each n)" << endl;

    runAll<int>("int", csv);
    runAll<double>("double", csv);

    cout << endl << "Saved to results.csv" << endl;
    return 0;
}
```

- `srand((unsigned)time(0));` — «заводим» генератор случайных чисел текущим временем, иначе при каждом запуске были бы одни и те же массивы.
- **Демонстрация шаблонов:** `insertionSort(a, 10)` сортирует массив `int`, `mergeSort(b, 5)` — массив `double`. Это одни и те же функции: компилятор сам подставляет `T = int` и `T = double`.
- `ofstream csv("results.csv");` — создаём файл результатов, первая строка — заголовки столбцов (type;data;n;insertionSort;mergeSort;std::sort).
- `fixed << setprecision(3)` — выводить числа с 3 знаками после запятой.
- `runAll<int>("int", csv);` — все замеры на int, `runAll<double>("double", csv);` — все замеры на double.

8. Что такое шаблонная функция

`template <typename T>` перед функцией говорит компилятору: «`T` — это какой-то тип, подставишь его сам». Функцию пишем один раз, а компилятор при вызове создаёт нужную версию:

```
int a[10];    insertionSort(a, 10);   // компилятор создаёт insertionSort<int>
double b[5];  mergeSort(b, 5);        // компилятор создаёт mergeSort<double>
runAll<double>("double", csv);        // и все замеры для double
```

В программе шаблонные **все** функции, которые работают с массивами: сортировки, заполнение, проверка, печать и замер. Поэтому один и тот же код проводит эксперимент и для `int`, и для `double`. Нешаблонная осталась только `printHeader` — она печатает шапку таблицы и с массивами не работает.

Без шаблона пришлось бы писать отдельную функцию для каждого типа (как `using Array = array<int, SIZE>` в практиках — там сортировки работали только для int и только для 10 элементов).

Требование к типу `T`: для него должны работать операторы `<` и `<=`. Подойдут int, double, char, string и свои структуры, если для них определить эти операторы.

Когда шаблон передаётся как указатель на функцию (в `measure` или `runTests`), компилятору не из чего вывести `T`, поэтому тип пишем явно: `insertionSort<T>`, `fillRandom<T>`.

9. Схемы алгоритмов (ГОСТ 19.701-90)

Файлы `uml/insertionSort.uxf` и `uml/mergeSort.uxf` открываются в UMLet.
Обозначения:

- овал — начало / конец (терминатор);
- прямоугольник — действие (процесс);
- ромб — условие (решение), выходы подписаны «да» / «нет»;
- граница цикла (фигура со срезанными углами, похожая на трапецию) — цикл `for`: сверху «начало цикла» (срезаны верхние углы) с переменной и условием, снизу «конец цикла» (срезаны нижние углы), тело цикла — между ними. Возвратные стрелки не нужны: повтор подразумевается;
- цикл `while` в insertionSort нарисован ромбом-условием со стрелкой возврата к нему;
- прямоугольник с двойными боковыми линиями — вызов функции (предопределённый процесс).

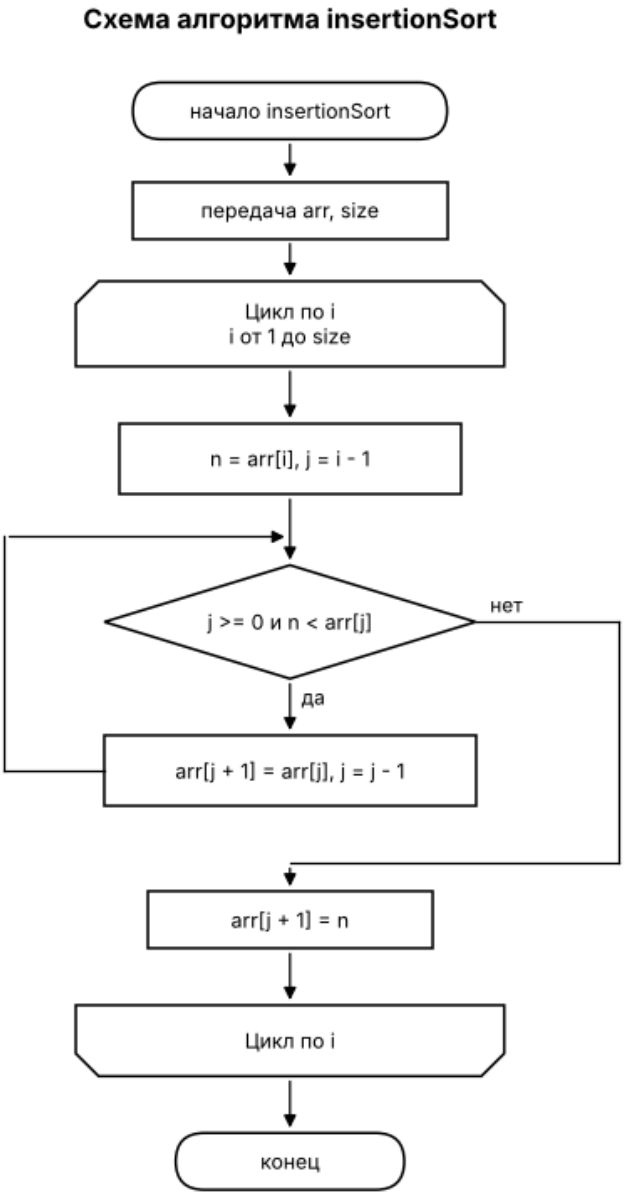


Схема алгоритма mergeSort

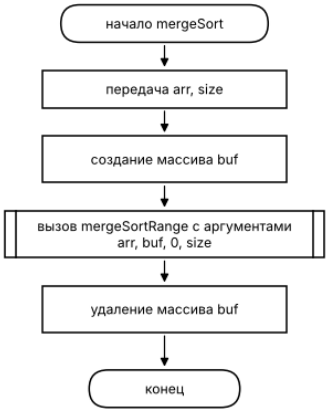
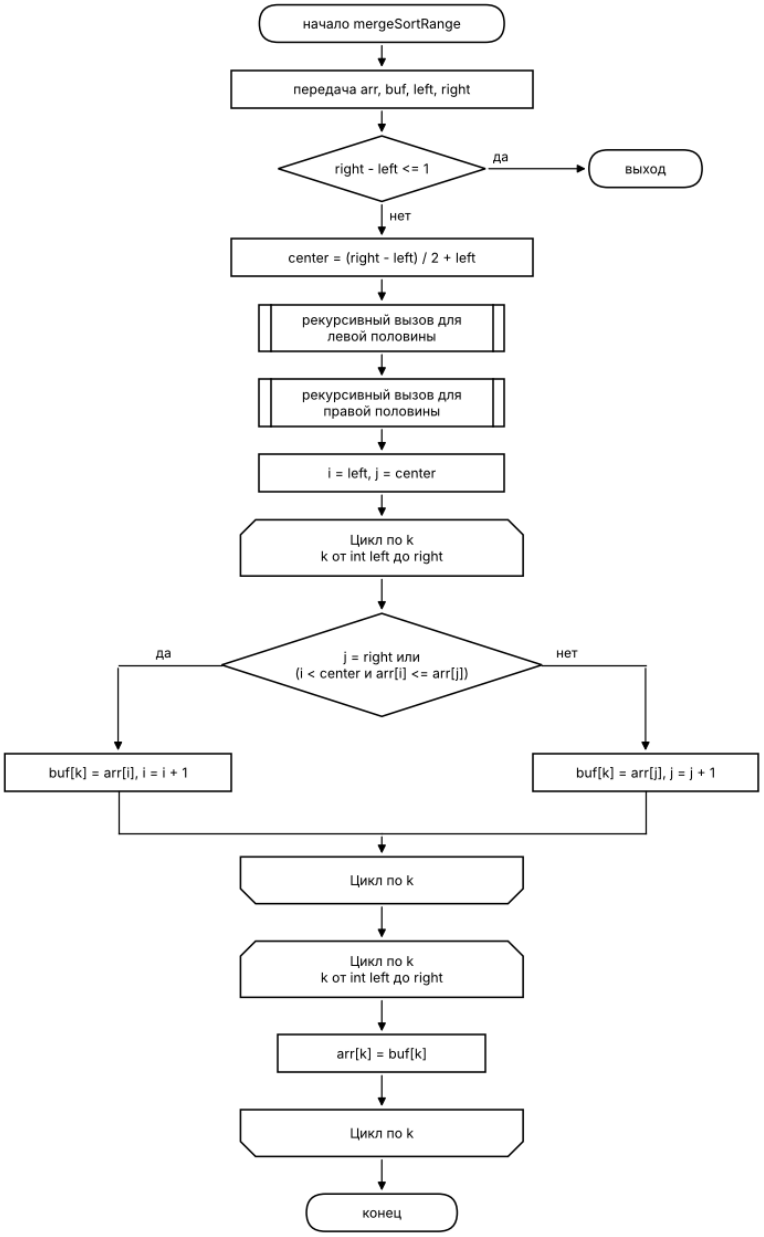


Схема алгоритма mergeSortRange



10. Результаты эксперимента

Среднее время в миллисекундах, 10 массивов на каждое n, сборка с оптимизацией (аналог Release). На другом компьютере абсолютные цифры будут другими, но соотношения сохранятся.

int, случайные массивы (п. 4)

n	insertionSort	mergeSort	std::sort
1 000	0.145	0.116	0.050
2 000	0.459	0.153	0.106
5 000	2.626	0.434	0.278
10 000	9.405	0.921	0.695
20 000	50.579	2.101	1.298
30 000	113.970	3.377	2.134
40 000	167.345	4.231	2.764
50 000	253.223	5.277	3.326
100 000	—	10.757	6.738
200 000	—	23.025	14.234
500 000	—	62.282	38.090
1 000 000	—	145.876	89.627

int, массивы из 10 упорядоченных фрагментов (п. 5)

n	insertionSort	mergeSort	std::sort
1 000	0.141	0.047	0.040
2 000	0.366	0.079	0.065
5 000	2.007	0.202	0.159
10 000	8.727	0.418	0.350
20 000	38.837	0.872	0.730
30 000	80.940	1.440	1.151
40 000	137.759	1.858	1.495
50 000	205.447	2.340	2.146
100 000	—	5.240	4.342
200 000	—	10.291	7.879
500 000	—	26.171	20.058
1 000 000	—	51.225	38.533

double, случайные массивы (п. 4)

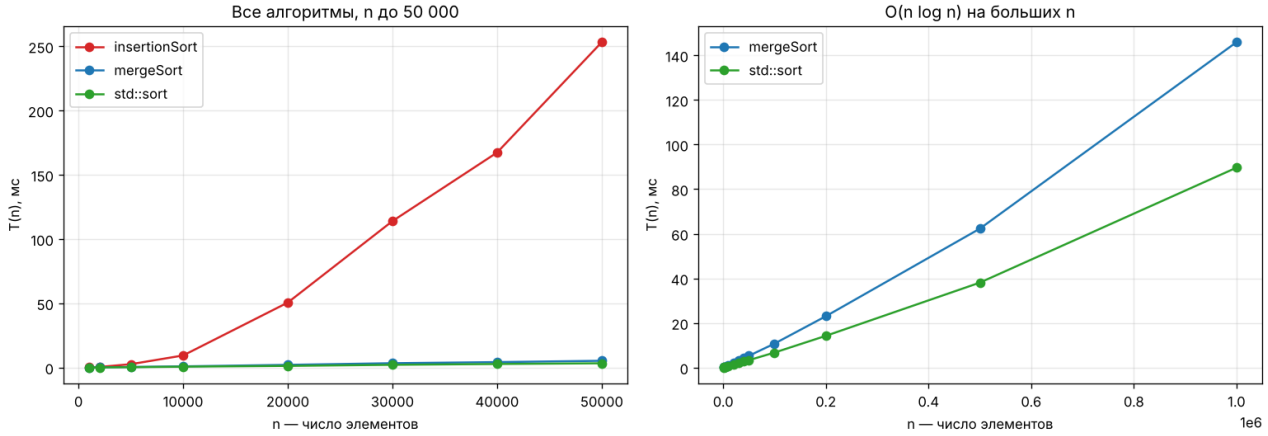
n	insertionSort	mergeSort	std::sort
1 000	0.172	0.088	0.055
2 000	0.574	0.158	0.099
5 000	3.295	0.415	0.267
10 000	14.571	1.057	0.836
20 000	54.904	2.030	1.279
30 000	132.724	3.420	2.046
40 000	223.393	4.396	2.725
50 000	360.000	5.760	3.530
100 000	—	11.735	7.156
200 000	—	25.762	16.572
500 000	—	69.038	41.264
1 000 000	—	149.935	89.718

double, массивы из 10 упорядоченных фрагментов (п. 5)

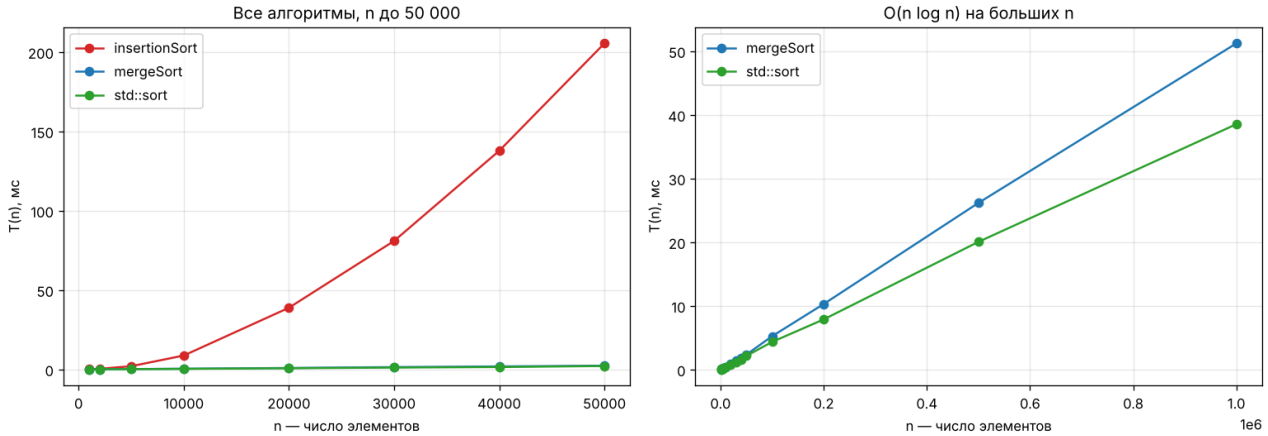
n	insertionSort	mergeSort	std::sort
1 000	0.127	0.042	0.032
2 000	0.503	0.085	0.064
5 000	3.344	0.242	0.175
10 000	15.291	0.603	0.441
20 000	54.484	1.135	0.857
30 000	114.943	1.627	1.175
40 000	221.276	2.332	1.584
50 000	324.173	2.573	1.840
100 000	—	5.731	4.025
200 000	—	12.581	8.564
500 000	—	31.093	20.669
1 000 000	—	69.658	43.454

Графики T(n)

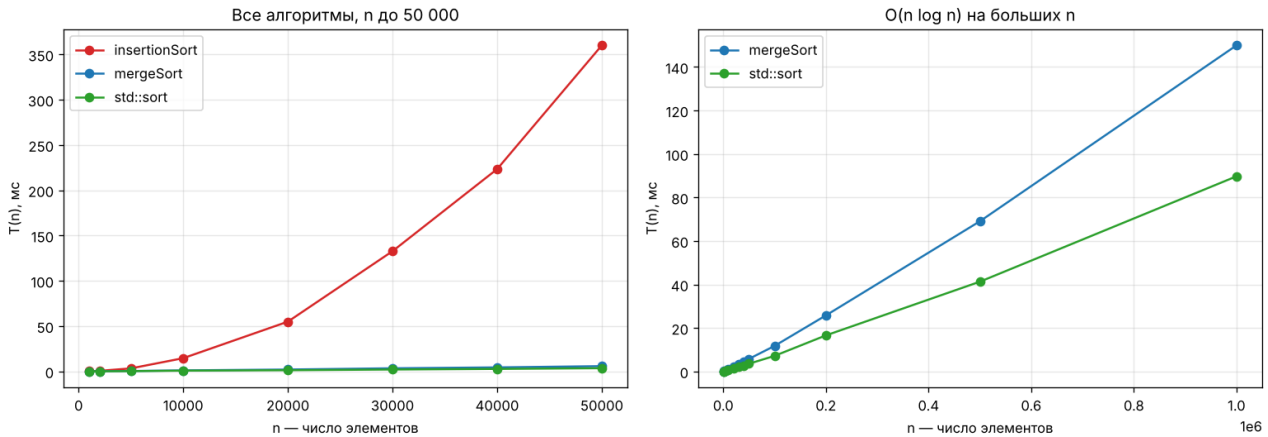
int: случайные массивы

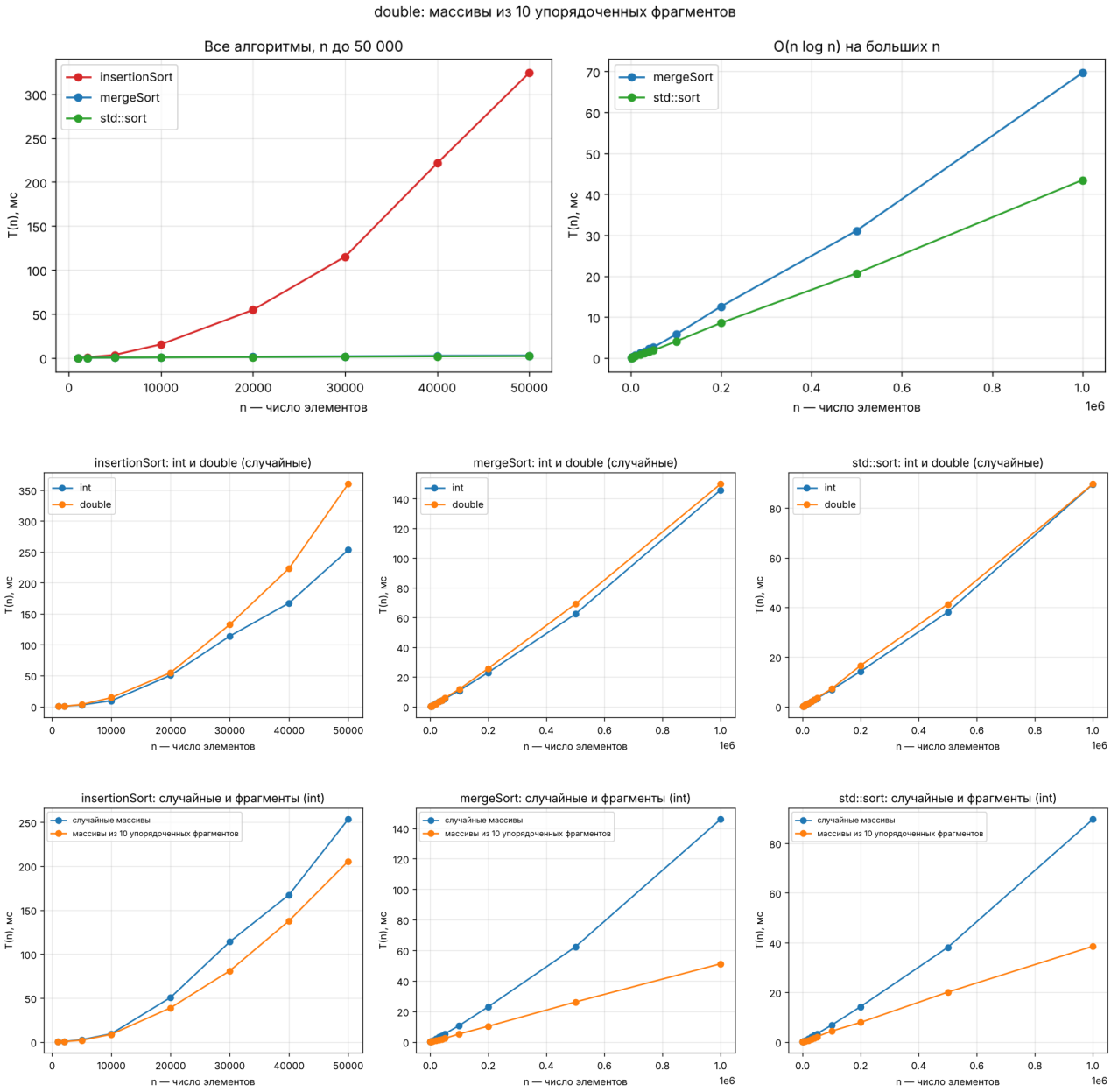


int: массивы из 10 упорядоченных фрагментов



double: случайные массивы




double: массивы из 10 упорядоченных фрагментов
Все алгоритмы, n до 50 000
insertionSort
mergeSort
std::sort
T(n), мс
n — число элементов
O(n log n) на больших n
mergeSort
std::sort
T(n), мс
n — число элементов
1e6
insertionSort: int и double (случайные)
int
double
T(n), мс
n — число элементов
mergeSort: int и double (случайные)
int
double
T(n), мс
n — число элементов
1e6
std::sort: int и double (случайные)
int
double
T(n), мс
n — число элементов
1e6
insertionSort: случайные и фрагменты (int)
случайные массивы
массивы из 10 упорядоченных фрагментов
T(n), мс
n — число элементов
mergeSort: случайные и фрагменты (int)
случайные массивы
массивы из 10 упорядоченных фрагментов
T(n), мс
n — число элементов
1e6
std::sort: случайные и фрагменты (int)
случайные массивы
массивы из 10 упорядоченных фрагментов
T(n), мс
n — число элементов
1e6

11. Выводы

- **Квадратичный рост подтверждается.** При удвоении n время insertionSort растёт примерно в 4 раза. Для double: 10 000 → 20 000 даёт ×3.8, 20 000 → 40 000 — ×4.1. Для int отдельные точки дают от ×3.3 до ×5.4 — это погрешность замера, в среднем около 4. На графике — парабола.
- **Рост n log n подтверждается.** 500 000 → 1 000 000: mergeSort ×2.34 (int) и ×2.17 (double). Теория: $2 \cdot \log(10^6) / \log(5 \cdot 10^5) \approx 2.1$. На графике почти прямая.
- **Разница огромная и растёт с n.** При n = 50 000 insertionSort медленнее mergeSort в 48 раз (int) и 62 раз (double). При n = 1000 — всего в 1.2 раза.
- **std::sort быстрее нашей mergeSort в ~1.6 раза.** Сложность у них одинаковая, отличается константа: std::sort работает на месте, без буфера и без копирования buf → arr на каждом уровне.
- **int и double.** Тип элементов не меняет порядок роста — графики имеют ту же форму. Меняется только константа: insertionSort на double медленнее в 1.4 раза (double занимает 8 байт вместо 4, а вставки постоянно двигают элементы в памяти). mergeSort и std::sort почти не замечают разницы (×1.03 и ×1.00).
- **Упорядоченные фрагменты.** insertionSort ускоряется мало — на 19% (int) и 10% (double): фрагменты перемешаны между собой, и элементам всё равно приходится сдвигаться через чужие куски. Сложность по-прежнему $O(n^2)$. mergeSort быстрее в 2.8 раза, std::sort — в 2.3 раза (int). Операций столько же, но сравнения внутри фрагментов предсказуемы, и процессор быстрее их выполняет.
- **Итог:** для больших массивов выбор алгоритма важнее любой оптимизации кода и типа данных. $O(n^2)$ годится только для маленьких или почти отсортированных массивов. На практике лучше `std::sort`.

12. Вопросы на защите и ответы

Что такое O-нотация?

Оценка того, как растёт время работы при росте n. Константы и младшие слагаемые отбрасываются: $n^2/4 + 3n + 10 \to O(n^2)$. Важен только характер роста при больших n.

Почему выбрали вставки, а не пузырёк или выбор?

Все три — $O(n^2)$, но вставки на практике быстрее: делают только сдвиги, без обменов. На почти отсортированных данных они работают за O(n), а сортировка выбором всегда делает $\sim n^2/2$ сравнений.

Почему слияние, а не быстрая сортировка Хоара?

У Хоара худший случай $O(n^2)$ при неудачном опорном элементе. Слияние всегда O(n log n). Плата — O(n) дополнительной памяти под буфер.

Что такое устойчивая сортировка? Ваши устойчивые?

Устойчивая — равные элементы сохраняют исходный порядок. Обе наши устойчивые: в insertionSort стоит `n < arr[j]` (равные не сдвигаются), в слиянии `arr[i] <= arr[j]` (при равенстве берём из левой половины). std::sort неустойчива, устойчивая — std::stable_sort.

Сколько дополнительной памяти нужно?

insertionSort — O(1), одна переменная. mergeSort — O(n) на буфер плюс O(log n) на стек рекурсии.

Какая глубина рекурсии у mergeSort?

$\log_2 n$: для миллиона элементов около 20 уровней. Переполнения стека не будет.

Лучший и худший случай для ваших сортировок?

insertionSort: лучший — отсортированный массив, O(n); худший — массив по убыванию, $O(n^2)$. mergeSort: всегда O(n log n).

Почему время растёт не ровно в 4 или 2 раза?

Мешают кэш процессора, фоновые программы, погрешность таймера. Поэтому берём среднее по 10 массивам. Кроме того, для n log n множитель при удвоении чуть больше 2 из-за роста log n.

Зачем копировать массив перед каждой сортировкой?

Чтобы все три алгоритма сортировали одинаковые данные. Иначе второй получил бы уже отсортированный массив.

Зачем isSorted?

Это проверка правильности. Быстрая, но неправильная сортировка ничего не стоит.

Почему insertionSort не запускается после 50 000?

При n = 1 000 000 время выросло бы в $(10^6/5 \cdot 10^4)^2 = 400$ раз: около 3 минут на один массив, а массивов 40 (10 × 2 вида данных × 2 типа). Для сравнения хватает размеров до 50 000.

Почему запускаете в Release, а не в Debug?

В Debug компилятор не оптимизирует код и добавляет проверки. Время получается в разы больше и искажает сравнение алгоритмов.

Что такое указатель на функцию?

`void (*sortFunc)(T[], int)` — переменная, хранящая адрес функции. Через неё можно вызвать любую функцию с такой сигнатурой. Так одна функция measure мерит все три сортировки.

Почему массивы через new, а не int arr[1000000]?

Локальный массив лежит на стеке, а стек по умолчанию 1 МБ. Миллион int — 4 МБ, миллион double — 8 МБ, программа бы упала. Память из new берётся из кучи, где места много. Освобождается через delete[].

Как сортировка слиянием обрабатывает массив нечётной длины?

center = (right − left)/2 + left округляется вниз, поэтому половины отличаются на 1 элемент. Например, для 5 элементов получится 2 и 3 элемента. Алгоритм от этого не ломается.